Automated Quality Assessment of LLM Generated Unit Tests: A Home Assistant Case Study

Karri Rakesh Reddy

Status. Generated by `make all` from `results/*.csv`. Every number in the Results section is read from those files. The comparison with the blinded manual review (RQ2) is *pending manual review*. The behaviour list used for relevance and completeness is a *draft that needs a human check*.

Introduction

Large language models can write unit tests, and several systems report that a large share of the generated tests compile, run, and add coverage (Alshahwan et al. 2024; Schäfer et al. 2024; Lemieux et al. 2023). Coverage alone is a weak statement about test *quality*, however: a test suite can execute every line and still miss most defects. This study assesses LLM generated unit tests along several automatically measurable dimensions and checks the automatic scores against a blinded manual review.

The case study is the `threshold` integration of Home Assistant core (tag 2026.10.1), a small component with an entity, a configuration flow and a migration path. Its developers' tests form the human baseline and already reach complete line coverage, which is why branch coverage and mutation score are the main comparison measures.

We ask three research questions.

- **RQ1.** How do LLM generated tests compare with the developers' tests on executability, coverage, mutation score, traceability, maintainability (test smells), relevance and completeness?
- **RQ2.** How well do the automatic scores agree with a blinded manual review of a sample (Spearman's ρ, weighted Cohen's κ)?
- **RQ3.** How sensitive are generated tests to metamorphic changes of the prompt (paraphrase, renamed local variables, reordered functions, stripped comments), compared with the normal noise between repeated samples?

Related work

LLM based unit test generation. TestGen-LLM at Meta (Alshahwan et al. 2024) filters generated Kotlin tests through build, pass, and coverage-improvement checks before accepting them. TestPilot (Schäfer et al. 2024) evaluates LLM generated tests for JavaScript packages and uses failure feedback to repair them. CodaMosa (Lemieux et al. 2023) couples search based test generation with an LLM to escape coverage plateaus. ChatUniTest (Chen et al. 2024) is a framework for Java that follows a generate, validate, repair loop, and CoverUp (Altmayer Pizzorno and Berger 2025) drives Python test generation with coverage feedback. A survey (Wang et al. 2024) organises the wider field of software testing with LLMs.

Test quality beyond coverage. Mutation testing (Jia and Harman 2011) estimates fault detection ability by injecting small syntactic faults. Siddiq et al. (Siddiq et al. 2024) study LLM generated JUnit tests and report on test smells in them; PyNose (Wang et al. 2021) is a test smell detector for Python, which motivates our own AST based detector. Test-to-code traceability links have been recovered by combining several signals (White et al. 2020), an idea we reuse in a light form.

Robustness. ReCode (Wang et al. 2023) evaluates code generation models under semantics preserving perturbations of prompts. Metamorphic testing (Chen et al. 2018) provides the general idea of relating outputs of related inputs, which we apply to the generation prompt.

Method

Pipeline. The `testquality` tool (this repository) generates tests with the Anthropic Messages API, runs them in the Home Assistant checkout as a subprocess, and computes all metrics from the run artefacts. Every LLM call is logged in full (`data/runs/<run>/calls.jsonl`). The developers' tests are never shown to the model: a prompt contains only the integration's own source files and the tool aborts if a developer test name occurs in a prompt.

Generation. For each of three target modules (`binary_sensor.py`, `config_flow.py`, `__init__.py`) one prompt asks for a pytest module; every test must carry a docstring with `Behaviour:` and `Target:` lines. $k = 5$ samples per prompt are drawn to measure the noise floor. No sampling parameters are sent. The main model is `claude-sonnet-5-5`.

Metrics (full definitions in `docs/metrics.md`).

- *Executability*: collect rate, pass rate over three repeated runs, flaky rate, and failure types. Only stably passing tests are *valid* and count for every other metric.
- *Coverage*: line and branch coverage of the component (coverage.py with per-test contexts).
- *Mutation score*: mutmut 3.x on the component. We report the score over a fixed population of mutants (killed / all) and the conventional killed / (killed + survived).
- *Traceability*: the function suggested by the test name, the last component function entered before the first assertion (dynamic trace), and the declared `Target:` are compared and scored 0, 0.5 or 1.
- *Maintainability*: ten test smells with a custom AST detector.
- *Relevance and completeness*: a draft list of behaviours with code line ranges; a test is relevant if it exercises a listed behaviour, and completeness is the share of behaviours exercised.
- *Manual review (RQ2)*: 60 tests (20 developer, 40 LLM) in random order with the origin hidden, rated 1-5 on relevance, clarity of purpose, maintainability and oracle strength. Agreement is measured by Spearman's ρ and linearly weighted Cohen's κ (Cohen 1968).
- *Metamorphic sensitivity (RQ3)*: MR1 three paraphrases of the prompt; MR2 renamed local variables, MR3 reordered functions and methods, MR4 stripped comments in the shown source. Each transformation is proven meaning preserving by tests (AST equality up to renaming, equal definition sets). Variants are compared with the baseline samples by Mann-Whitney U (Mann and Whitney 1947), Cliff's δ (Cliff 1993), and by the Jaccard overlap of covered lines and of killed mutants relative to the overlap between baseline samples.

Setup

Home Assistant core is checked out at tag 2026.10.1 (branch `study-pinned`) with its own virtual environment. The tool runs in a separate environment (Python 3.12 or newer, `uv`). Generated tests are staged under `tests/components/threshold/_llm_generated/<run_id>/` and removed after each run; no other Home Assistant file is modified. Mutation testing runs in a scratch workspace outside the Home Assistant tree. Coverage uses the `ctrace` core of coverage.py, because the default core of Python 3.12 and newer drops per-test contexts. Model, prompt version, costs and all decisions are recorded in `docs/DECISIONS.md`.

Results

Key findings

- **Executability.** 15 of 15 LLM samples collect; 97.6 % of the 294 generated tests pass in all three runs (developer tests: 100.0 %); flaky rate 0.0 %.
- **Coverage and mutation (suite level).** Line coverage 100.0 % (developer) vs 99.6 % (union of the LLM samples); branch coverage 93.1 % vs 96.5 %; mutation score over the fixed 164 mutants 78.7 % vs 81.1 %.
- **Per target mutation score (global).** binary_sensor: 56.0 % per LLM sample vs 59.8 % for the developer file; config_flow: 5.4 % per LLM sample vs 47.6 % for the developer file; init: 50.0 % per LLM sample vs 54.3 % for the developer file.
- **Traceability.** Mean name-vs-coverage score 0.378 (developer) vs 0.301 (LLM); the LLM's declared `Target:` scores 0.533, and 92.0 % of valid LLM tests carry a parsable declaration.
- **Test smells.** Share of valid tests with at least one smell: 98.3 % (developer) vs 44.2 % (LLM); smell density 2.496 vs 0.662 per test.
- **Relevance and completeness (draft behaviour list).** Relevance 99.1 % (developer) vs 91.6 % (LLM); LLM completeness 100.0 % of the 38 behaviours reachable by the developer tests.
- **Metamorphic sensitivity.** pending (results/mr_metrics.csv missing).
- **Agreement with manual review (RQ2).** pending manual review.

RQ1: generated tests versus developer tests

Executability

Suite	Units	Collect rate	Tests	Pass rate (mean of 3)	Stable pass rate	Flaky rate
developer	3/3	100.0 %	115	100.0 %	100.0 %	0.0 %
main_sonnet55_v1	15/15	100.0 %	294	97.6 %	97.6 %	0.0 %
mr_mr1a_binary_sensor	5/5	100.0 %	128	100.0 %	100.0 %	0.0 %
mr_mr1b_binary_sensor	5/5	100.0 %	160	100.0 %	100.0 %	0.0 %
mr_mr1c_binary_sensor	5/5	100.0 %	117	100.0 %	100.0 %	0.0 %

Failure types of non-passing tests

Suite	Error type	Tests
main_sonnet55_v1	RuntimeError	6
main_sonnet55_v1	AssertionError	1

Coverage and mutation score (suite level)

Developer rows per target are the matching developer file; the LLM rows are the union over the k = 5 samples of that target.

Target / scope	Suite	Tests passing	Line cov.	Branch cov.	Mutation score (global)	Mutation score (covered)
all	developer	115	100.0 %	93.1 %	78.7 %	78.7 %
all	LLM (union of k=5)	287	99.6 %	96.5 %	81.1 %	81.1 %
target:binary_sensor	developer	97	86.2 %	75.9 %	59.8 %	62.0 %
target:binary_sensor	LLM (union of k=5)	145	86.6 %	77.6 %	59.2 %	82.9 %
target:config_flow	developer	9	84.2 %	56.9 %	47.6 %	59.1 %
target:config_flow	LLM (union of k=5)	80	76.8 %	51.7 %	12.2 %	62.5 %
target:init	developer	9	65.8 %	25.9 %	54.3 %	70.6 %
target:init	LLM (union of k=5)	62	74.4 %	53.4 %	57.9 %	75.4 %

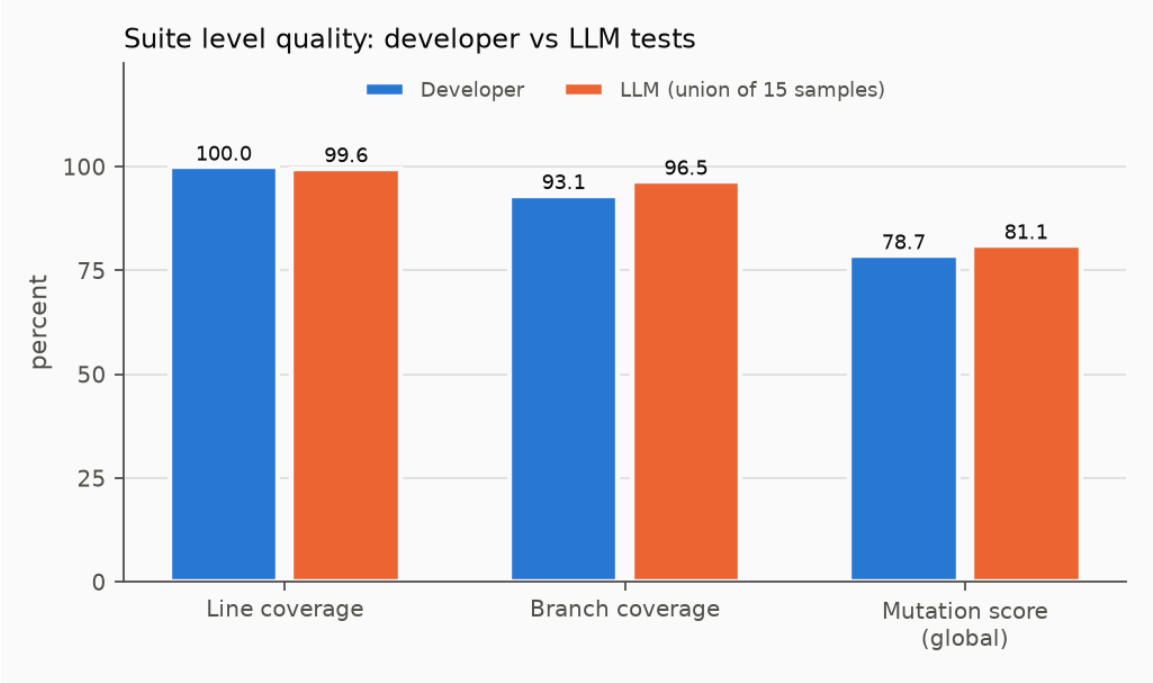


Figure 1: Suite level quality of developer and LLM tests.

Per file / per sample

Target	Developer file	LLM samples (mean ± sd, k=5)
binary_sensor - valid tests	97	29.0 ± 4.9
binary_sensor - line cov.	86.2 %	86.6 % ± 0.0 %
binary_sensor - branch cov.	75.9 %	77.6 % ± 0.0 %
binary_sensor - mutation score (global)	59.8 %	56.0 % ± 2.9 %
config_flow - valid tests	9	16.0 ± 2.1
config_flow - line cov.	84.2 %	48.9 % ± 15.6 %
config_flow - branch cov.	56.9 %	13.1 % ± 21.6 %
config_flow - mutation score (global)	47.6 %	5.4 % ± 3.8 %
init - valid tests	9	12.4 ± 3.2
init - line cov.	65.8 %	68.2 % ± 0.9 %
init - branch cov.	25.9 %	33.4 % ± 4.0 %
init - mutation score (global)	54.3 %	50.0 % ± 2.0 %

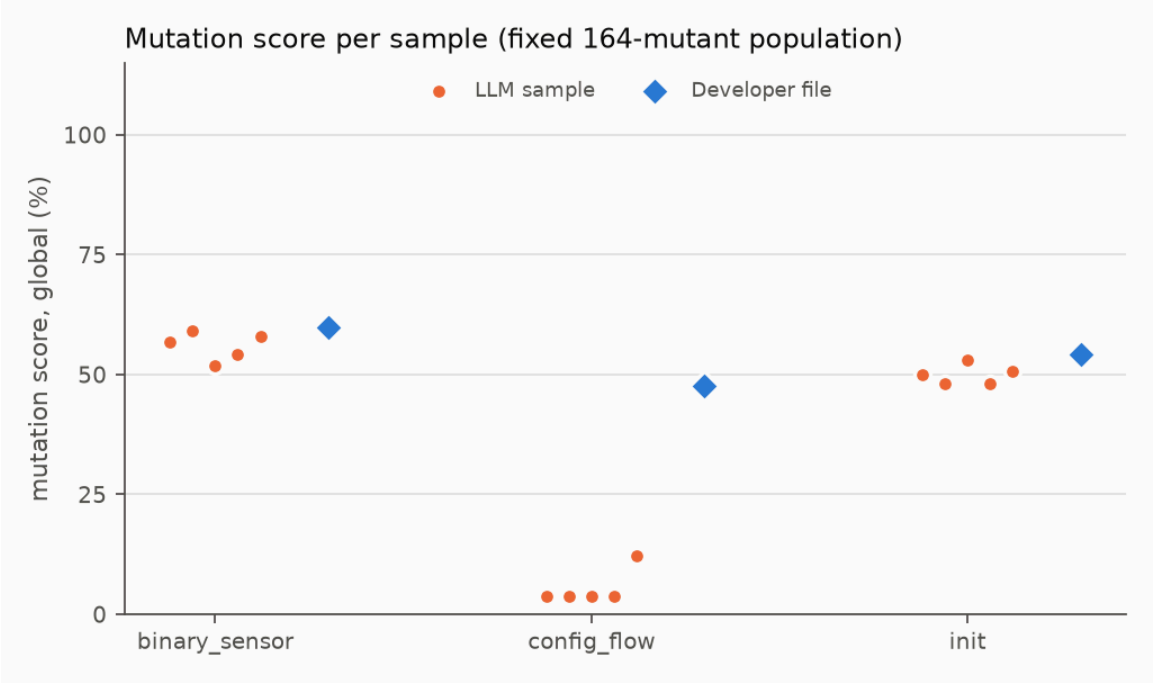


Figure 2: Mutation score of each LLM sample and of the developer file per target.

Traceability, smells, relevance, completeness

Behaviour list is a DRAFT (needs human check); completeness is relative to the behaviours the developer tests reach.

Suite	Trace (name target)	Trace (declared target)	Declared present	Smelly tests	Smell density	Relevance	Completeness
developer	0.378	n/a	n/a	98.3 %	2.496	99.1 %	100.0 %
LLM	0.301	0.533	92.0 %	44.2 %	0.662	91.6 %	100.0 %

Test smells (share of valid tests with the smell)

Smell	developer	LLM
assertion_roulette	97.4 %	20.2 %
duplicate_assert	6.1 %	3.8 %
magic_number	60.0 %	27.2 %
conditional_logic	79.1 %	0.0 %
no_assertion	0.0 %	5.6 %
try_except	0.0 %	0.0 %
sleepy_test	0.0 %	0.0 %
redundant_print	0.0 %	0.0 %
eager_test	0.0 %	0.0 %
empty_test	0.0 %	0.0 %

RQ2: agreement with the manual review

RQ2 agreement with manual review
pending manual review (manual_review/sample_rated.csv not present).

RQ3: metamorphic sensitivity

RQ3 metamorphic sensitivity
pending: results/mr_metrics.csv not found.

Overlap of killed mutants between variants and baseline versus the noise floor.
Figure 3: Overlap of killed mutants between variants and baseline versus the noise floor.

Threats to validity

Construct validity. Traceability is defined operationally; entity properties that Home Assistant reads on every state write (for example `extra_state_attributes`) often become the "last call", which lowers the score for tests of either origin. Relevance and completeness rest on a draft behaviour list written by the assistant that has not yet been reviewed by a human, and a test is mapped to a behaviour by coverage, which does not prove that the behaviour is verified. Mutation tracking in mutmut is function level, so a unit's tests are only run for mutants of functions they execute, and the conventional score flatters narrow suites; we therefore also report the fixed denominator score. Module-level behaviours (schemas executed at import time) cannot be attributed to single tests and are excluded.

Internal validity. The blinding of the manual review is imperfect: naming and structure can reveal the origin even after docstrings are normalised. Failing or flaky tests are excluded from all quality metrics, which favours suites with few tests. The tool was developed together with the

analysis, and parts of it (smell thresholds, mapping threshold) were fixed before looking at group differences but are still design choices.

External validity. One small integration of one project, one model family, one prompt template, and $k = 5$ samples per prompt: results may not carry over to other components, languages or models. The model may have seen Home Assistant tests during training, so the isolation of the developer tests applies to the prompt, not to the model's knowledge. The metamorphic experiment uses a single target module and five samples per variant, so its statistical power is low; "no significant difference" must not be read as "equivalent".

Conclusion validity. Rates over parametrised tests weigh heavily parametrised files more. Agreement statistics for RQ2 depend on one rater and on the binning of automatic scores.

Conclusion

The tool chain measures executability, coverage, mutation score, traceability, test smells, relevance and completeness for LLM generated and developer tests of a Home Assistant component and supports a metamorphic robustness analysis. The findings above summarise the measured differences; the comparison with manual judgement (RQ2) remains pending until the ratings exist. Interpreting the differences requires the caveats listed under threats to validity.

References

Alshahwan, Nadia, Jubin Chheda, Anastasia Finogenova, et al. 2024. "Automated Unit Test Improvement Using Large Language Models at Meta." *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion '24)*, 185–96. https://doi.org/10.1145/3663529.3663839.

Altmayer Pizzorno, Juan, and Emery D. Berger. 2025. "CoverUp: Effective High Coverage Test Generation for Python." *Proceedings of the ACM on Software Engineering* 2 (FSE): 2897–919. https://doi.org/10.1145/3729398.

Chen, Tsong Yueh, Fei-Ching Kuo, Huai Liu, et al. 2018. "Metamorphic Testing: A Review of Challenges and Opportunities." *ACM Computing Surveys* 51 (1): 1–27. https://doi.org/10.1145/3143561.

Chen, Yinghao, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. "ChatUniTest: A Framework for LLM-Based Test Generation." *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion '24)*, 572–76. https://doi.org/10.1145/3663529.3663801.

Cliff, Norman. 1993. "Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions." *Psychological Bulletin* 114 (3): 494–509. https://doi.org/10.1037/0033-2909.114.3.494.

Cohen, Jacob. 1968. "Weighted Kappa: Nominal Scale Agreement Provision for Scaled Disagreement or Partial Credit." *Psychological Bulletin* 70 (4): 213–20. https://doi.org/10.1037/h0026256.

Jia, Yue, and Mark Harman. 2011. "An Analysis and Survey of the Development of Mutation Testing." *IEEE Transactions on Software Engineering* 37 (5): 649–78. https://doi.org/10.1109/TSE.2010.62.

Lemieux, Caroline, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. "CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-Trained Large Language Models." *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 919–31. https://doi.org/10.1109/ICSE48619.2023.00085.

Mann, H. B., and D. R. Whitney. 1947. "On a Test of Whether One of Two Random Variables Is Stochastically Larger Than the Other." *The Annals of Mathematical Statistics* 18 (1): 50–60. https://doi.org/10.1214/aoms/1177730491.

Schäfer, Max, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. "An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation." *IEEE Transactions on Software Engineering* 50 (1): 85–105. https://doi.org/10.1109/TSE.2023.3334955.

Siddiq, Mohammed Latif, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinícius Carvalho Lopes. 2024. "Using Large Language Models to Generate JUnit Tests: An Empirical Study." *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE '24)*, 313–22. https://doi.org/10.1145/3661167.3661216.

Wang, Junjie, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. "Software Testing with Large Language Models: Survey, Landscape, and Vision." *IEEE Transactions on Software Engineering* 50 (4): 911–36. https://doi.org/10.1109/TSE.2024.3368208.

Wang, Shiqi, Zheng Li, Haifeng Qian, et al. 2023. "ReCode: Robustness Evaluation of Code Generation Models." *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 13818–43. https://doi.org/10.18653/v1/2023.acl-long.773.

Wang, Tongjie, Yaroslav Golubev, Oleg Smirnov, Jiawei Li, Timofey Bryksin, and Iftekhar Ahmed. 2021. "PyNose: A Test Smell Detector for Python." *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 593–605. https://doi.org/10.1109/ASE51524.2021.9678615.

White, Robert, Jens Krinke, and Raymond Tan. 2020. "Establishing Multilevel Test-to-Code Traceability Links." *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE '20)*, 861–72. https://doi.org/10.1145/3377811.3380921.